Do it MySQL 입문 6장

문자열 함수

1. **문자열 더하기: CONCAT**

```
CONCAT(string_value1, ..., string_valueN)
(e.g.)
SELECT CONCAT('a', 'pineapple', 'is', 'apple')

SELECT CONCAT(first_name, ',', last_name) as customer_name
FROM customer;
```

2. **데이터 형 변환: CAST, CONVERT**

- 사용자가 임의로 형 변환하는 과정: 명시적 형변환
- 범위를 벗어나 자동으로 데이터 형이 변환되는 경우 암시적 형변환

```
SELECT CAST(column AS DATA_TYPE) FROM 테이블;
SELECT CONVERT(column, DATA_TYPE) FROM 테이블;
```

📌 CAST, CONVERT에 사용 가능한 데이터 타입

- binary(BINARY)
- character(CHAR)
- DATE
- DATETIME
- TIME
- DECIMAL
- JSON
- NCHAR
- SIGNED / UNSIGNED

ℹ️ 앞선 함수를 사용할 때, `CHAR(5)` 와 같이 문자열의 길이 지정 가능 ⇒ 이때, char에 지정한 값이 문자열의 길이보다 작으면 문자열이 잘리게 됨!

- 자료형의 범위를 넘어가면 오버플로우 발생 ⇒ 데이터 타입 변경!

3. **NULL 문자열 대체: IFNULL, COALESCE**

- NULL은 어떠한 연산 작업에도 NULL 반환 → 치환 필

```
IFNULL(column, 대체할 값)
COALESCE(column1, column2, ...)
```

4. **문자열 공백 제거: TRIM**

- `LTRIM`: 문자열의 왼쪽 공백 제거
- `RTRIM` : 문자열의 오른쪽의 공백 제거
- `TRIM`: 양쪽 공백 제거

5. **특정 문자까지의 길이 반환: POSITION**

- 지정한 특정 문자까지의 길이 반환

```
SELECT 'I LOVE!! MySQL', POSITION('!' IN 'I Love!! MySQL');
```

6. **지정 길이만큼 문자열 반환: LEFT, RIGHT**

- 문자열 왼쪽이나 오른쪽에서 시작하여 정의한 위치까지 문자열 반환
- 문자열 위치 시작 값: 1

```
SELECT 'I LOVE!! MySQL', LEFT('I LOVE!! MySQL', 2), RIGHT
('I LOVE!! MySQL', 2);
```

7. **지정 범위 만큼 문자열 반환: SUBSTRING**

- 2번째 인자는 시작범위, 3번째 인자는 시작 위치로부터 반환할 문자 개수

```
SELECT 'I LOVE! MySQL', SUBSTRING('I Love! MySQL', 4, 2);
```

8. **특정 문자를 다른 문자로 변경: REPLACE**

```
SELECT first_name, REPLACE(first_name, 'A', 'C')
FROM customer
WHERE first_name LIKE 'A%';
```

날짜 함수

```
-- 접속 중인 DB 서버의 날짜: CURRENT_DATE()
-- 시간: CURRENT_TIME()
-- 날짜 + 시간: CURRENT_TIMESTAMP() OR NOW()

SELECT CURRENT_DATE(), UTC_DATE(), UTC_TIME(), UTC_TIMESTAM
P(), NOW();

-- 날짜 더하거나 빼기: DATEADD()
-- 날짜 형식(date), 더하거나 뺄 대상 날짜 숫자(interval), 년또는
월 또는 일(expr unit)

SELECT NOW(), DATE_ADD(NOW(), INTERVAL 1 YEAR);
SELECT NOW(), DATE_ADD(NOW(), INTERVAL -1 YEAR);
```

```sql
-- 날짜 차이 구하기: DATEDIFF()
SELECT DATEDIFF('TIMESTAMP1', 'TIMESTAMP2')

-- 두 날짜 사이의 기간을 다양한 포맷으로 확인: TIMESTAMPDIFF
SELECT TIMESTAMPDIFF(MONTH, 'TIMESTAMP1', 'TIMESTAMP2');


-- 날짜 요일, 일, 년, 월 변환: DAYNAME, MONTH, YEAR, ...
```

집계 함수

```sql
-- 조건에 맞는 데이터 세기: COUNT()
-- 반환 값: BIGINT 범위
-- 특정 열만 지정하면 해당 열에 대한 NULL 값은 제외
SELECT COUNT(*) FROM customer;
SELECT COUNT(*), COUNT(store_id), COUNT(DISTINCT store_id)
from customer;

-- 데이터 합 구하기: SUM()
SELECT SUM(amount) FROM payment;
SELECT customer_id, SUM(amount) FROM payment GROUP BY custo
mer_id;

-- 데이터 평균: AVG()
-- NULL 값 무시 & DISTINCT 문을 조합하여 중복값 무시하고, 고유한
값에 대해서만 적용 가능
SELECT AVG(amount) FROM payment;
SELECT customer_id, AVG(amount) FROM payment GROUP BY custo
mer_id;

-- 최댓값: MAX(). 최솟값: MIN()
SELECT MIN(amount), MAX(amount) FROM payment;

-- 부분합, 총합 구하기: ROLLUP
-- GROUP BY ROLLUP 조합하여 사용
SELECT customer_id, staff_id, SUM(amount)
FROM payment
```

```
GROUP BY customer_id, staff_id WITH ROLLUP;
```

수학 함수

```
-- 0과 1 사이의 실수형 난수 구하기
SELECT RAND(인자);
```

순위 함수

```
-- 순위 함수: ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE()
-- PARTITION 옵션을 함께 사용해 사용자가 지정한 그룹에 따라 그룹내
순위를 부여 가능
-- 유일한 값으로 순위 부여: ROW_NUMBER()
SELECT ROW_NUMBER() OVER (order by amount desc, customer_id
ASC) as num,
customer_id,
amount
FROM ~~

-- 같은 순위 개수를 고려하여 순위 부여: RANK()
-- 같은 순위의 경우 같은 값 부여
RANK () OVER ([partition by clause] order_by_clause

-- 그룹화하여 순위 부여: NTILE()
-- 인자로 지정ㅇ한 값만큼 데이터 행을 그룹화 하여 그룹별 순위 부여
SELECT NTILE(100) OVER (order by amount desc) as num, ...
```

분석 함수